

etcd 在大规模 AI 集群中的性能极限挑战与内核级突破

演讲者：汪扬 / 蚂蚁超算容器平台



Content 目录

01 AI时代的需求变化

02 工程破局

03 内核攻坚

04 社区前沿



Part 01

AI 时代的需求变化

社区标准与企业需求的错配



需求变化 与 规模鸿沟

- AI 需求驱动：
 - Kube AgentSandbox
 - 管控面 到 数据面 的转变
 - 大模型预训练：
 - 并行协同
 - 高性能网络
 - 拓扑感知调度



支持 5000 节点



GKE

支持 65000 节点

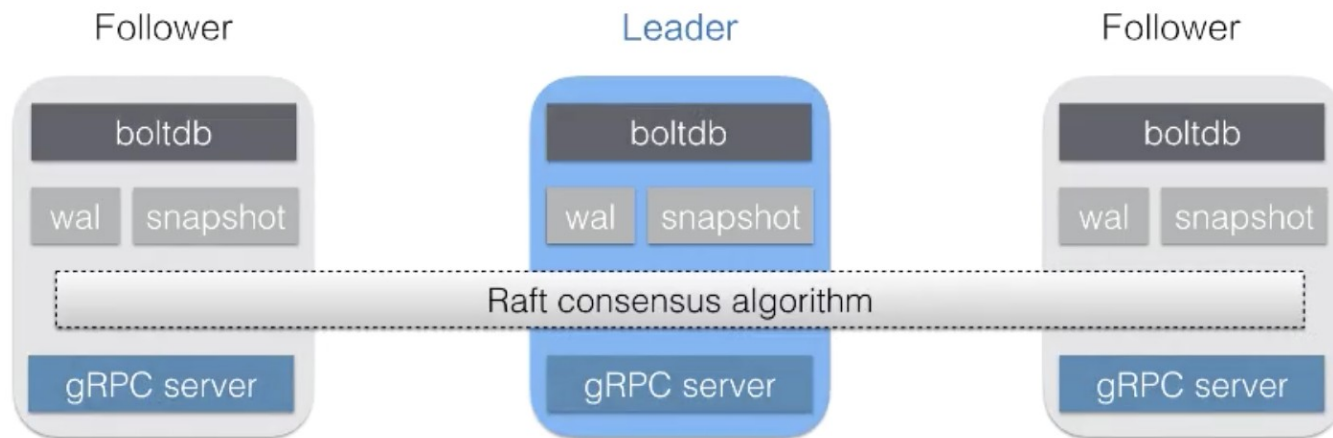
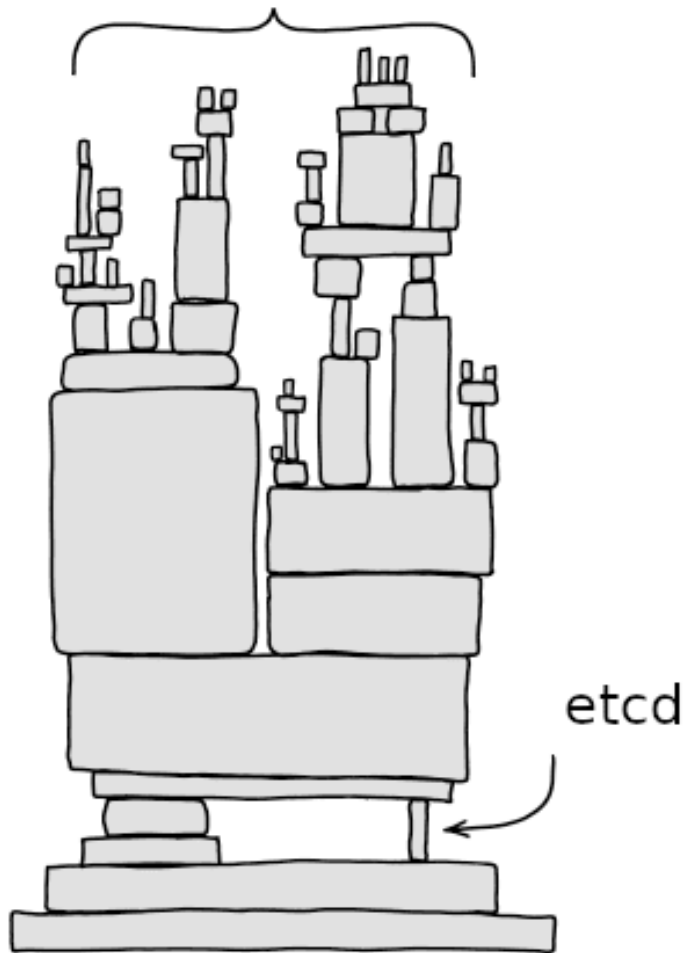


Amazon EKS

支持 100000 节点



All Cloud Native Infrastructure



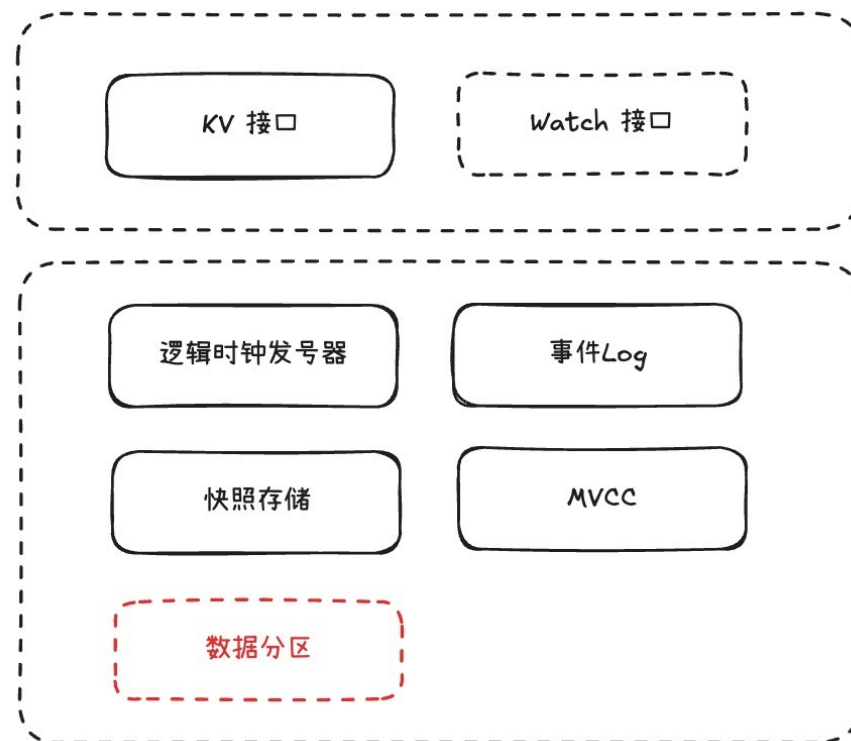
• Etcd 特点：

- KV 存储 + 事件通知
- 无法横向扩展
- 适合读多写少的负载
- Revision 全局连续单调递增
- 顺序依次 apply 日志



GKE 与 EKS 的解法

- GKE
 - 存储基于 spanner
- EKS
 - 卸载 raft 到 journal
 - 内存数据库
 - 资源维度 Partition + 动态 repartitioning
- kubebrain
 - 单点写，读放大
 - 卸载 raft, 写并发控制 到 TiKV
 - 单点 Watch, 内存事件 RingBuffer, 数据空洞



Part 02

工程化破局

资源维度的数据流量拆分



低垂果实：数据与流量拆分

- 流量分布

- 更新最频繁资源：Lease
- 容易失控资源：Event
- 高保资源：Pod

- 拆分技术基础：

- APIServer 已支持静态映射
- 工程化解决拆分问题

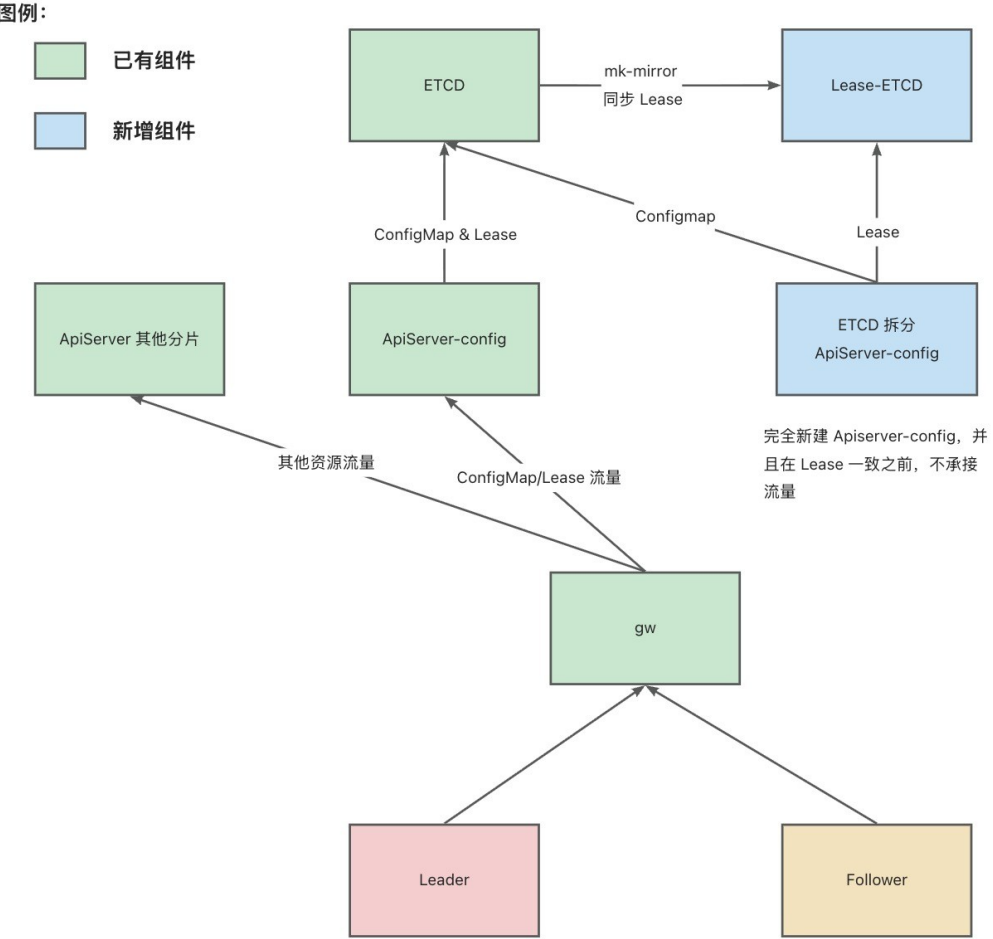
	max ▾	current
leases	459 K	458 K
pods	70.8 K	66.6 K
nodes	60.8 K	58.3 K
nodemetrics	59.1 K	58.9 K
noderesourceusages	52.3 K	51.6 K
workflowtasksets	48.2 K	3.24 K
events	46.0 K	43.1 K
noderesources	40.8 K	36.0 K

```
--etcd-servers-overrides strings
```

Per-resource etcd servers overrides, comma separated. The individual override format: group/resource#servers, where servers are URLs, semicolon separated. Note that this applies only to resources compiled into this server binary. e.g.
"/pods#http://etcd4:2379;http://etcd5:2379,/events#http://etcd6:2379"



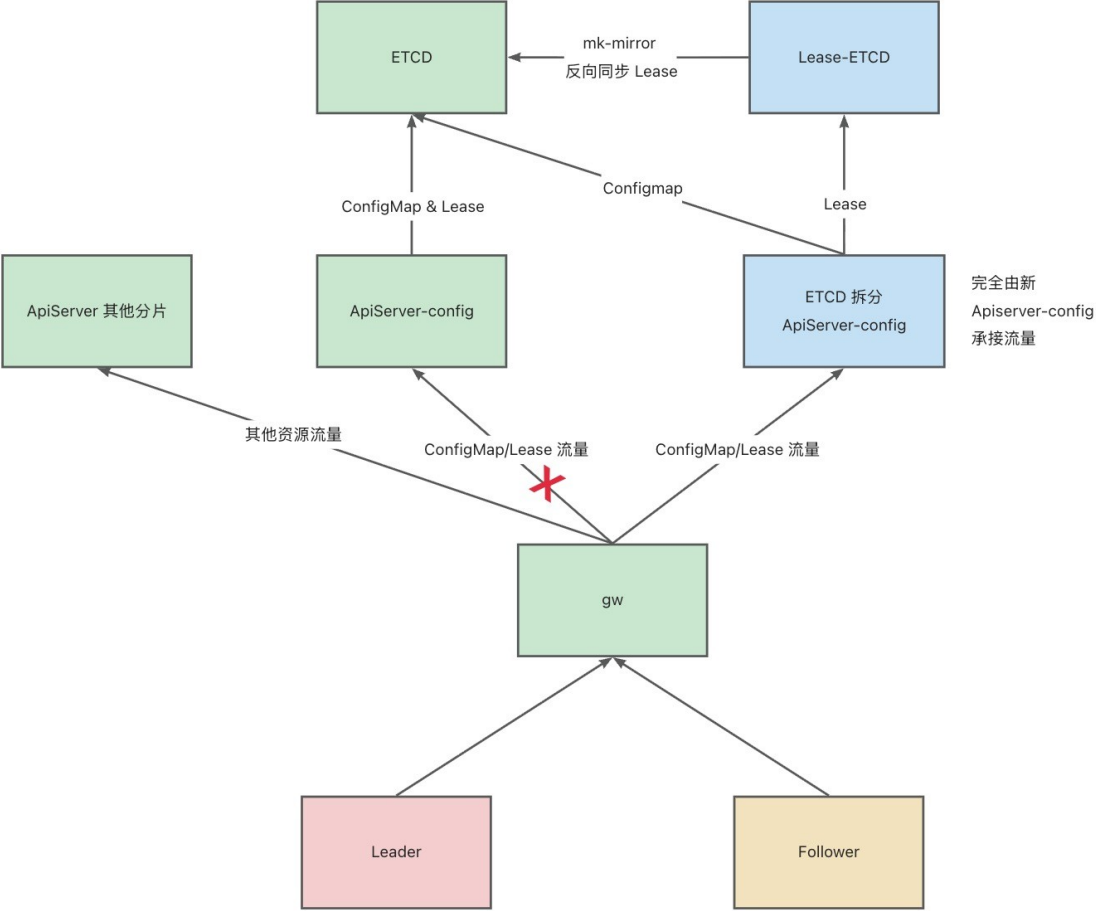
数据与流量拆分方案



当 Lease-ETCD 数据追上 ETCD 后, 执行流量切换



发现问题无损回滚



Part 03

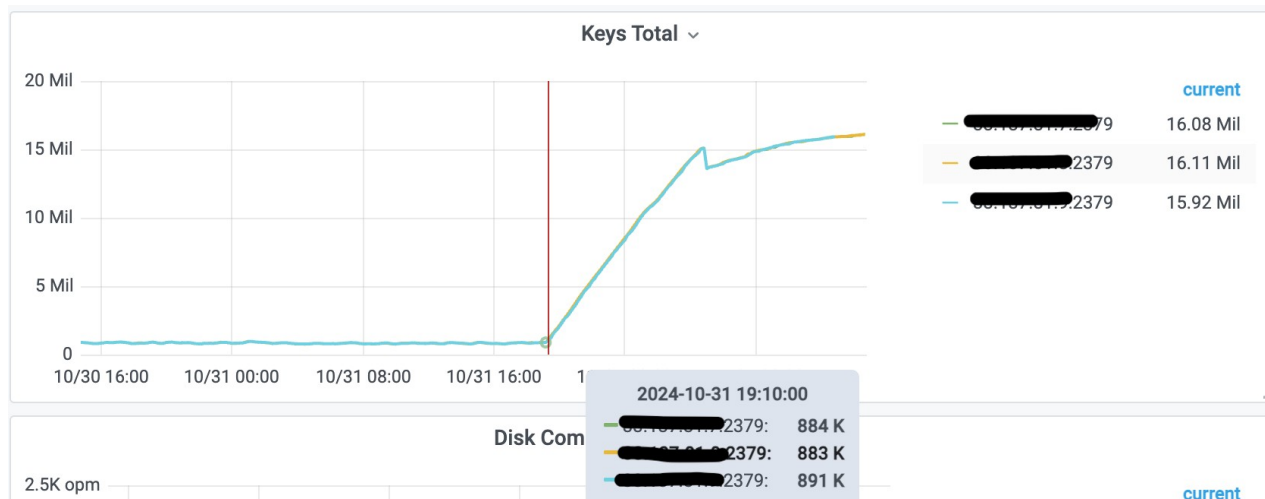
深入内核

稳定性和性能提升



极端场景下的 Lease 怪像

- APIServer 对 Event 的使用方式：
 - 每分钟申请一个 etcd Lease ID , TTL=30min
 - 复用 LeaseID 写入 k8s Event 对象
 - 30min 后 Event 对象自动被删除



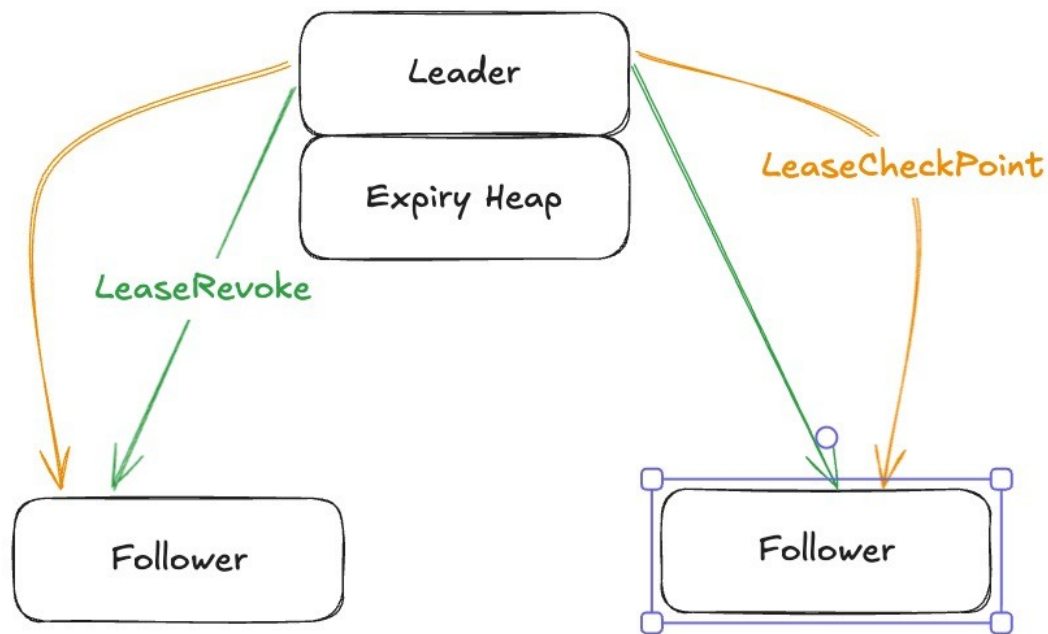
问题一：Leader 切换后为何 lease 停止淘汰？

问题二：集群崩溃的根因是什么？



深入理解 etcd Lease

- TTL 的约束：保证 Lease 存活时间不小于 TTL
- Leader 每 500ms 计算剩余 TTL 触发淘汰 (expiry 最小堆)
- --enable-lease-checkpoint, 每 5min 同步一次



根因

- etcd apply 速率小于请求 commit 速率，会直接拒绝请求
- LeaseRevoke 请求也被拒绝
- Key 数量积累，性能进一步恶化，陷入恶性循环

```
func (s *EtcdServer) processInternalRaftRequestOnce(ctx context.Context, r pl
    ai := s.getAppliedIndex()
    ci := s.getCommittedIndex()
    if ci > ai+maxGapBetweenApplyAndCommitIndex {
        return nil, ErrTooManyRequests
    }
```

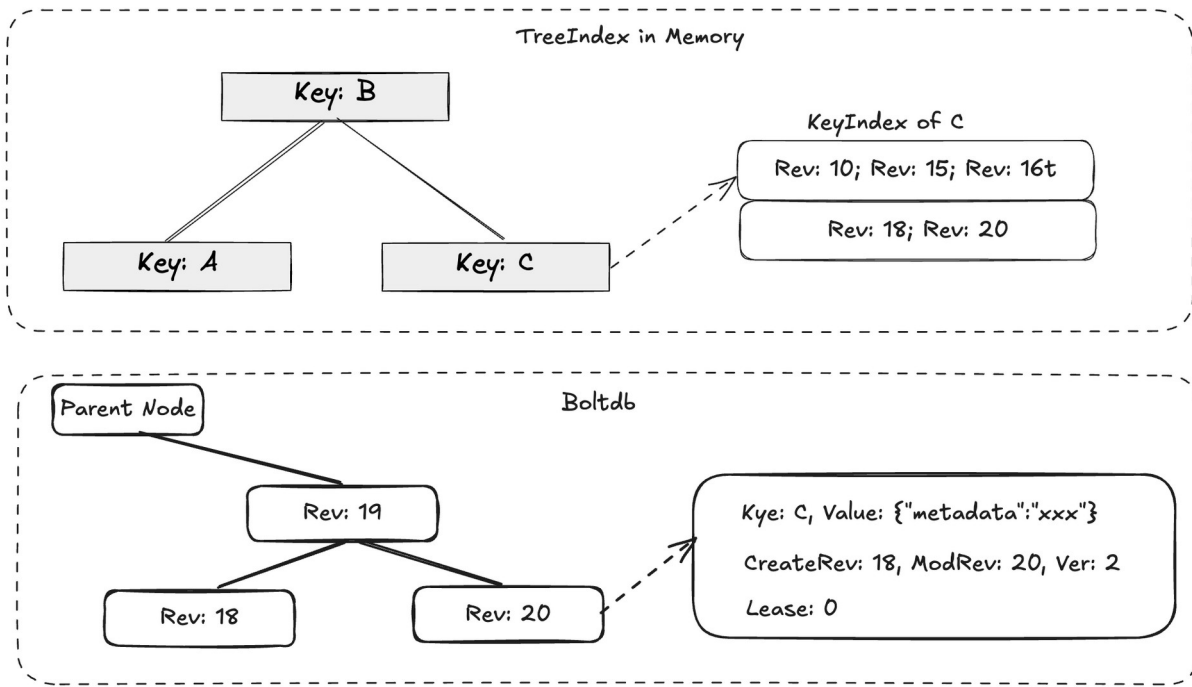
5000
Xiang Li, 8 years ago



Compaction 性能问题

Compaction 对数据量大的场景不友好

1. **锁粒度过粗**：遍历过程持有写锁，可能是不必要的。
2. **无效休眠**：每次遍历 CompactionBatchLimit 数量的 revision 后，都会 sleep CompactionSleepInterval 时长。这可能存在大量不必要的 sleep，因为遍历的 revisions 范围中没有需要清理的历史数据。

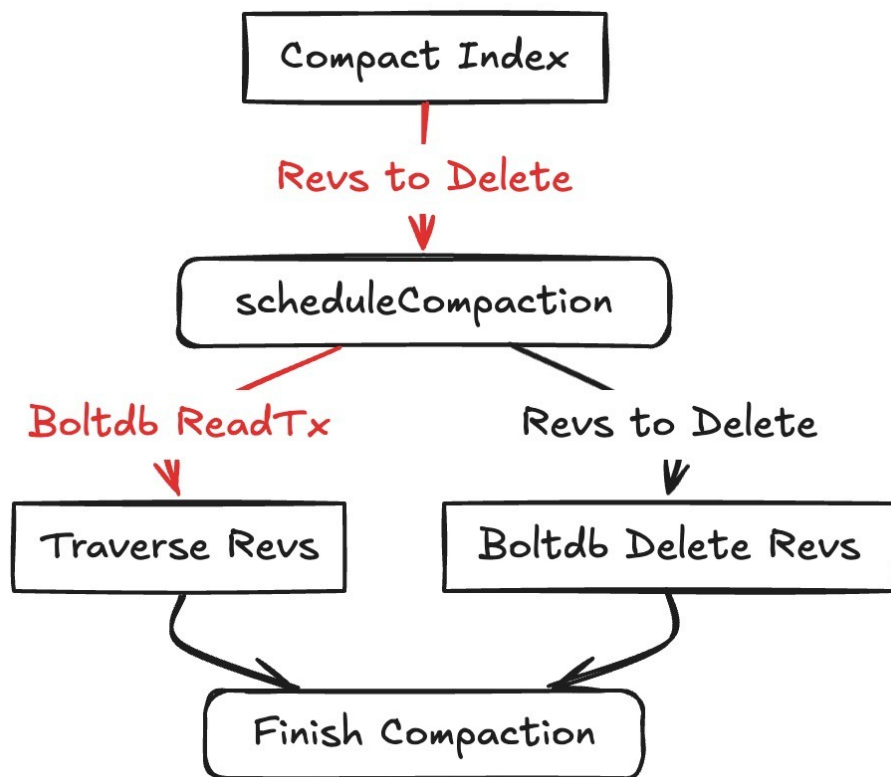


```
func (s *store) scheduleCompaction(compactMainRev, prevCompactRev int64) (KeyValueHash, error) {  
    // treeIndex compaction  
    keep := s.kvindex.Compact(compactMainRev) ← Compact treeIndex  
    h := newKVHasher(prevCompactRev, compactMainRev, keep)  
  
    for {  
        tx := s.b.BatchTx()  
        tx.LockOutsideApply() ← 写锁  
        // 遍历 revisions  
        keys, values := tx.UnsafeRange(schema.Key, last, end, int64(batchNum))  
        for i := range keys {  
            rev = BytesToRev(keys[i])  
            if _, ok := keep[rev]; !ok {  
                // 清理数据  
                tx.UnsafeDelete(schema.Key, keys[i])  
            }  
            // 计算哈希  
            h.WriteKeyValue(keys[i], values[i])  
        }  
  
        if len(keys) < batchNum {  
            UnsafeSetFinishedCompact(tx, compactMainRev)  
            tx.Unlock()  
            hash := h.Hash()  
            return hash, nil  
        }  
  
        tx.Unlock()  
        s.b.ForceCommit()  
  
        // sleep 10ms  
        select {  
        case <-time.After(s.cfg.CompactionSleepInterval):  
        case <-s.stopc:  
            return KeyValueHash{}, fmt.Errorf("interrupted due to stop signal")  
        }  
    }  
}
```

Compaction 优化方案

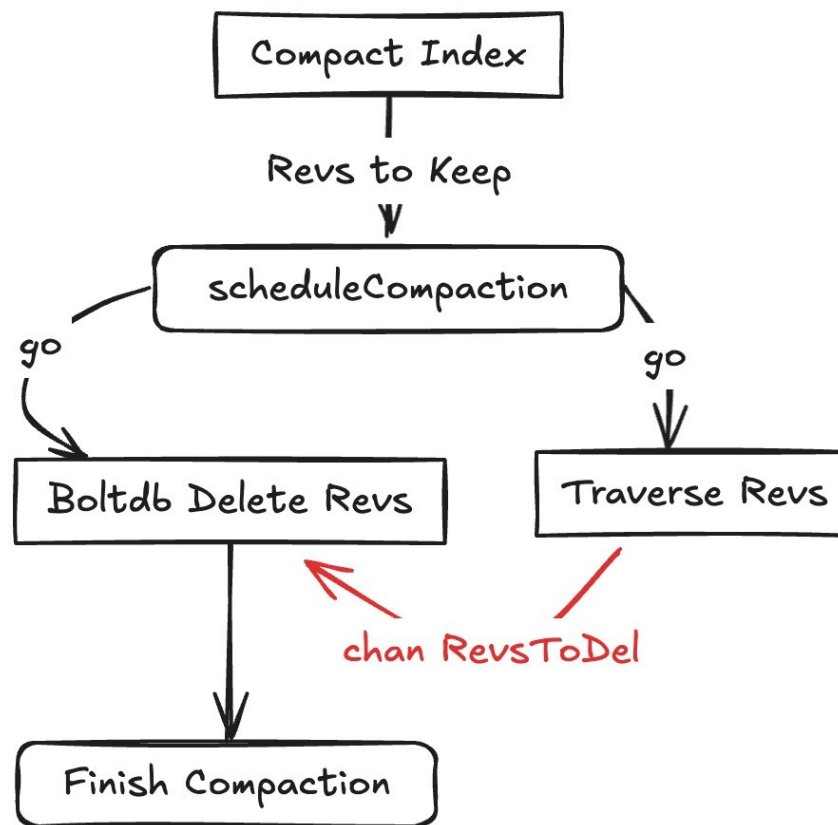
第一版修改

- 修改 Index 接口，在内存中计算出需要删除哪些数据
- 并行 KVHash 计算和清理历史数据，KVHash 依赖 boltdb 快照读
- 可选择执行 KVHash 遍历计算，对无需 KVHash 的场景非常高效



第二版修改

- 不修改 Index 接口，避免提升复杂度
- 分离 / 并行 KVHash 计算和清理历史数据，同时确保 KVHash 先于清理数据发生
- 减少读写锁持有范围，降低冲突



Compaction 优化效果

- 社区版本：总是遍历几乎全部 db 数据，每读取 1000 条 sleep 10ms，db 数据量对耗时有较大影响
- 第一版优化：把计算待删除数据的逻辑提前到了 Index 层，在 Compact 逻辑中并行 KVHash 和 清理数据，所以耗时是最优的。同时 KVHash 可选择关闭以获取最优效率，这个功能对大部分用户可能是不需要的。KVHash 依赖底层 boltdb 的快照读能力。耗时与写入数据量正相关。
- 第二版优化：不修改 Index 层，同样并行 KVHash 和 清理数据，清理数据依赖 KVHash 先扫描计算数据后通过 Channel 发送到 清理协程。保证顺序增加了一些耗时。KVHash 依旧是必选项，在高 TPS 写入负载下，能看到 TPS 略微下降。耗时与写入数据量正相关。

写入TPS	社区版耗时	第一版耗时	第二版耗时	耗时优化
1K/s	32.140923178s	6.051189144s	9.089513217s	72%
2K/s	38.630751905s	12.803279511s	14.736863272s	68%
5K/s	1m0.072742644s	31.387194197s	34.210398799s	48%
7K/s	1m20.295984944s	42.799126317s	47.139617551s	47%



探索与优化

数据压缩

- Benchmark Put 100w 数据，单个 Value 30K
- 使用 zstd 压缩算法 + 训练字典，压缩比大约 10: 1
- 写入性能 TPS 12261->25013, 上升 104%, 空间下降 81%
- 点查读性能：QPS 51531 -> 43392, 下降 16%

内存文件系统

- 数据量小，磁盘性能是瓶颈
- 快照 + 日志 = 状态
- 写入性能：
 - 1K 数据，39316/s P99 < 10ms
 - 4K 数据，TPS 6800 -> 28916, P99 < 30ms, TPS 提升 325%

Point-in-time Recovery

- 保存全量 WAL 文件
- 定时持久化快照
- 支持指定 Log Index 恢复数据

动态数据分区

- 分区前后 Revision 单调问题
- Server 端主动报错 CompactErr, 触发上层重新构建缓存
- Proxy 层路由更具弹性



Part 04

社区前沿



社区



- Features

- RangeStream: 优化 List 大量数据的内存消耗
- bbolt Compression: 减少 db 空间
- Remove explicit defrag: 去除显式 defrag
- Raft async write: 性能优化

- Bugfix

- Data Corruption: 并发 applySnapshot 与 defrag 导致数据错乱 [20271]
- Stale Read: 进程暂停导致线性一致读失效 [20418]



[20271] 并发 `applySnapshot` 与 `defrag` 导致数据错乱



复现步骤：

1. 使用较小 `snapshot-count` 参数 启动三节点集群
2. 通过 `gofail` 接口使 3rd 节点在 `OpenSnapshotBackend` 后 `sleep 15s`
3. 使用 `SIGSTOP` 暂停 3rd 节点
4. 1st 与 2nd 节点写入数据，并重启；目的是触发 leader 向 3rd 节点发送 `snap`
5. 使用 `SIGCONT` 恢复 3rd 节点，并调用 `defrag` 接口
6. Server 在 Step-2 获得了新的 be，但 Step-5 在 backend 内部删除了原先的 be 文件，导致数据错乱

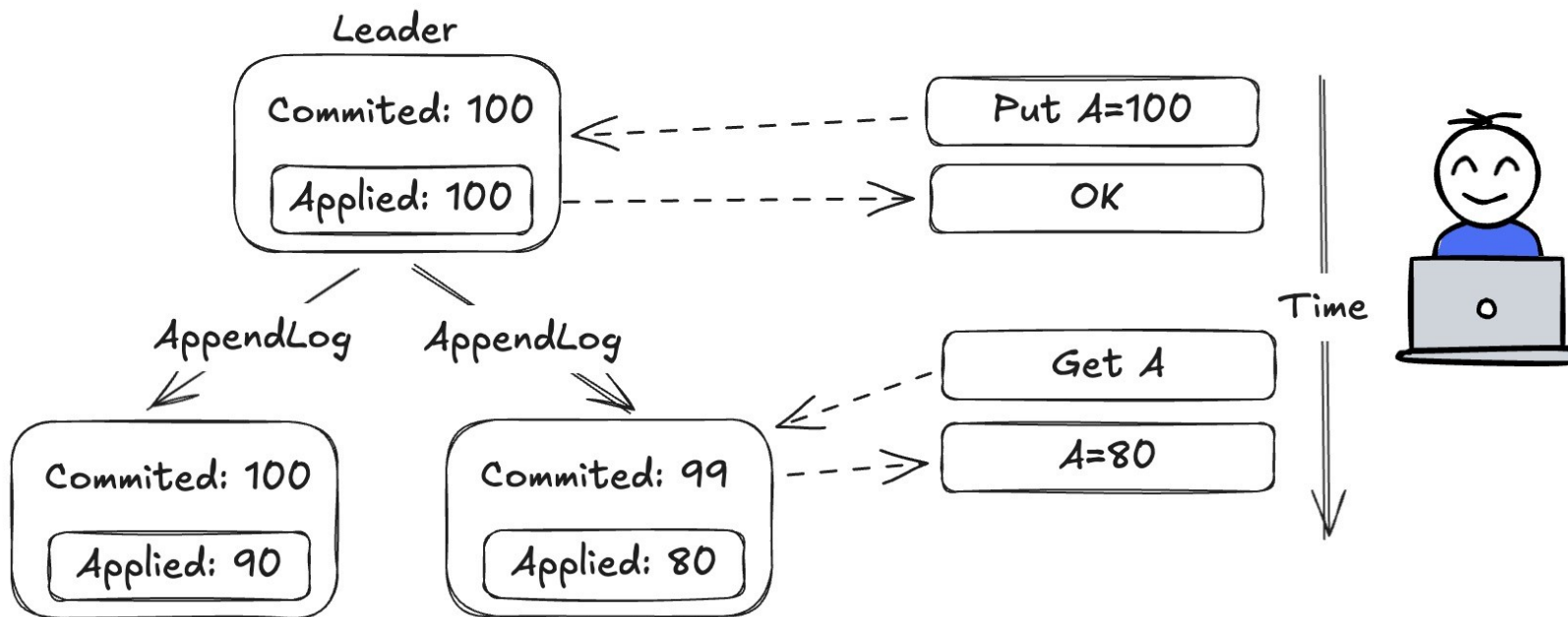
[fix #20271] protect backend usage from concurrent `applySnapshot` and `defrag` #20553

Merged [ahrtr](#) merged 1 commit into [etcd-io:main](#) from [silentred:fix-20271](#) on Sep 12, 2025

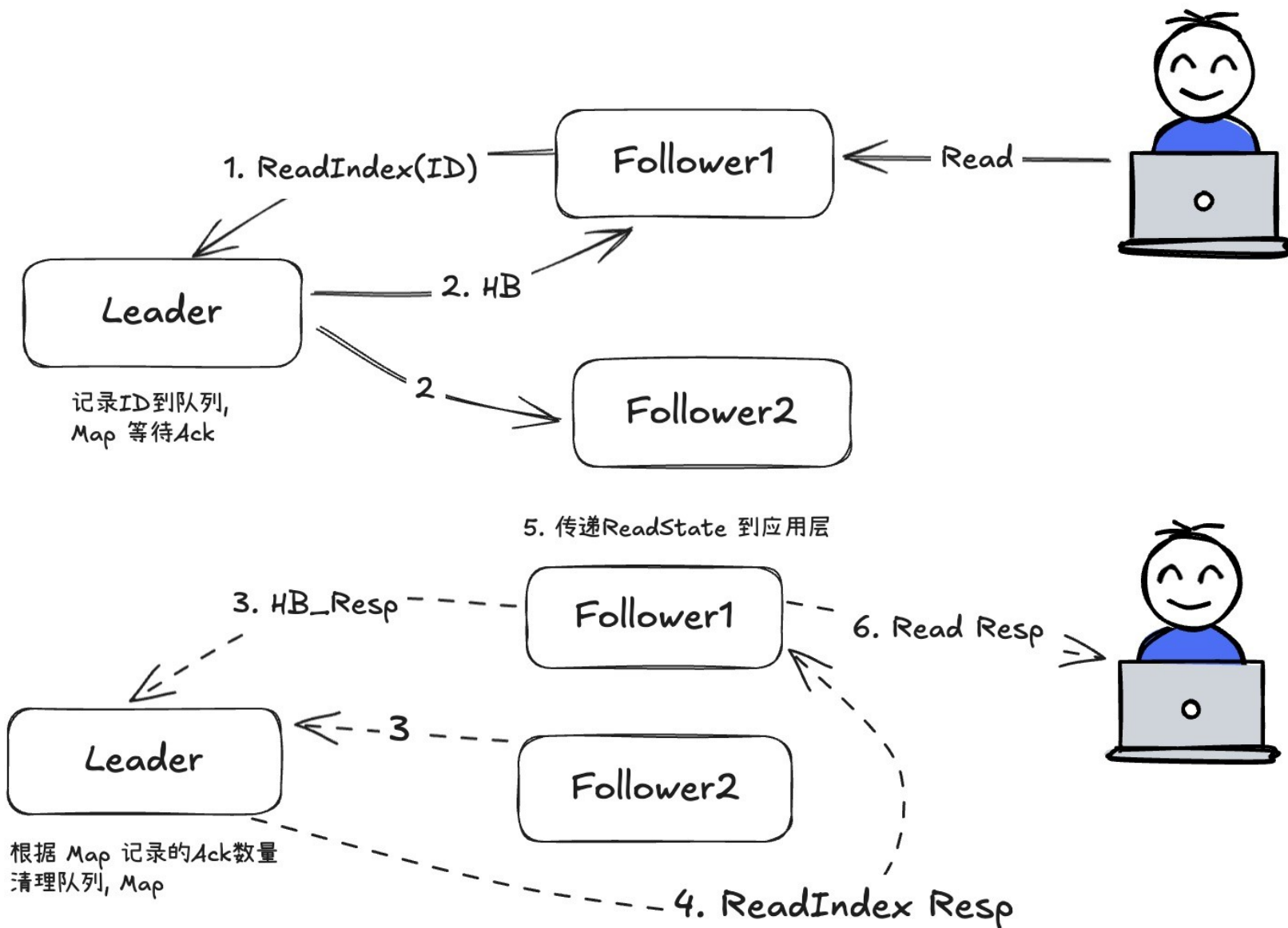


[20418] 进程暂停导致线性一致读失效

- 由于在 Raft 算法中，写操作成功仅仅意味着日志达成了一致（已经落盘），而并不能确保当前状态机也已经 apply 了日志。状态机 apply 日志的行为在大多数 Raft 算法的实现中都是**异步**的，所以此时读取状态机并不能准确反应数据的状态，很可能会读到过期数据。
- 一个较为简单通用的策略就是：每次读操作的时候记录此时集群的 committed index，当状态机的 apply index 大于或等于 committed index 时才读取数据并返回



[20418] 根因分析

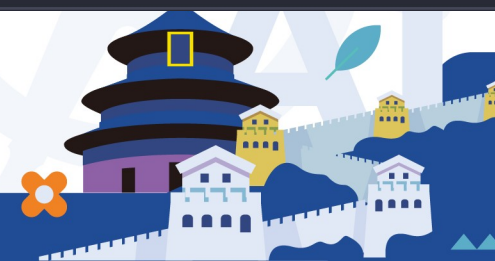


触发 **bug** 关键条件：

- Leader 进程暂停
- 集群发生切主
- 老 Leader 进程恢复
- 有新的 ReadIndex 请求进入
- 老 Leader 处理 Ack 逻辑有误

```
for _, okctx := range ro.readIndexQueue {
    i++
    rs, ok := ro.pendingReadIndex[okctx]
    if !ok {
        panic("cannot find corresponding read state from pending map")
    }
    rss = append(rss, rs)
    if okctx == ctx {
        found = true
        break
    }
}

if found {
    ro.readIndexQueue = ro.readIndexQueue[i:]
    for _, rs := range rss {
        delete(ro.pendingReadIndex, string(rs.req.Entries[0].Data))
    }
    return rss
}
```



更多 K8s 前沿的实践经验



关注后回复 K8s 即可获取专题链



Thanks.

